Assignment No.3

Title: Implementation of an N-Gram Language Model for Next Word Prediction

Aim

To implement an N-gram Language Model in Python that analyzes a text sequence and predicts the most probable next word.

Objective

1. To understand and generate N-grams (Unigram, Bigram, Trigram) from a given text dataset.
2. To preprocess text data by cleaning, tokenizing, and preparing it for analysis.
3. To calculate frequency counts and probabilities of word sequences in the dataset.
4. To implement the N-gram model in Python using appropriate data structures.
5. To predict the most probable next word based on the given input sequence.

Expected Outcomes

1. Successful generation of N-grams (Unigram, Bigram, Trigram) from a given text dataset.
2. Accurate calculation of word sequence frequencies and probabilities based on the training text.
3. Working Python implementation of the N-gram language model.
4. Ability to predict the most probable next word for a given input sequence.
5. Improved understanding of probabilistic language modeling concepts & text prediction techniques.

Theory

1.Introduction to Language Models

A Language Model (LM) is a probabilistic model that assigns a probability to a sequence of words. It helps in predicting the likelihood of a sentence or predicting the next word in a sequence.

For example:

Input: *"I love machine"*
Output Prediction: *"learning"*

Language models are widely used in:

- Text prediction
- Speech recognition
- Machine translation
- Chatbots
- Spell correction

2.Probability in Language Modeling

Given a sentence:

W = $w_1, w_2, w_3, \ldots, w_n$

The probability of the entire sentence is:

$$P(W) = P(w_1, w_2, w_3, \ldots, w_n)$$

Using the Chain Rule of Probability, this can be written as:

$$P(W) = P(w_1) \times P(w_2 \mid w_1) \times P(w_3 \mid w_1, w_2) \times \ldots \times P(w_n \mid w_1, \ldots, w_{n-1})$$

But calculating this is computationally expensive because it requires large data and memory.

3.N-Gram Assumption (Markov Assumption)

To simplify computation, N-gram models use the Markov assumption, which states:

The probability of a word depends only on the previous (N-1) words.

Instead of:

$$P(w_n \mid w_1, w_2, \ldots, w_{n-1})$$

We approximate as:

$$P(w_n \mid w_{n-1}, w_{n-2}, \ldots, w_{n-}(N-1))$$

This reduces complexity significantly.

4.Types of N-Grams

- **Unigram (N = 1)**

Probability depends only on the word itself.

$$P(w_n)$$

Example:

"I love AI"

Unigrams:
I, love, AI

- **Bigram (N = 2)**

Probability depends on previous one word.

$$P(w_n \mid w_{n-1})$$

Example:

I love AI

Bigrams:
(I, love), (love, AI)

- **Trigram (N = 3)**

Probability depends on previous two words.

$$P(w_n \mid w_{n-2}, w_{n-1})$$

Example:

I love AI

Trigrams:
(I, love, AI)

5.Probability Calculation

For a Bigram model:

$$P(w_n \mid w_{n-1}) = \frac{Count(w_{n-1}, w_n)}{Count(w_{n-1})}$$

For a Trigram model:

$$P(w_n \mid w_{n-2}, w_{n-1}) = \frac{Count(w_{n-2}, w_{n-1}, w_n)}{Count(w_{n-2}, w_{n-1})}$$

6.Text Preprocessing

Before building an N-gram model, text must be processed:

- Convert to lowercase
- Remove punctuation
- Tokenize (split into words)
- Remove unwanted characters

This ensures consistent probability calculation.

7.Advantages of N-Gram Model

1.Simple to implement
2. Easy probability computation
3. Works well with small datasets
4. Computationally efficient

Pseudocode

```
START
INPUT:
    Text corpus
    N (order of N-gram)
    Input sequence (for prediction)

STEP 1: Preprocess Text
    1. Convert text to lowercase
    2. Remove punctuation and special characters
    3. Tokenize text into words
    4. Store words in a list

STEP 2: Generate N-grams
    FOR i from 0 to length(words) - N
        context = words[i to i+N-2]
        next_word = words[i+N-1]
        Increase count of (context → next_word) in dictionary
    END FOR

STEP 3: Calculate Probabilities
    FOR each context in dictionary
        total_count = sum of all next_word counts for that context
        FOR each next_word in that context
            probability = count(context, next_word) / total_count
            Store probability
        END FOR
    END FOR
```

```
STEP 4: Predict Next Word
    1. Take last (N-1) words from input sequence as context
    2. If context exists in dictionary:
            Select next_word with highest probability
       Else:
            Return "No prediction available"

OUTPUT:
    Most probable next word

END
```

Results

```
Original Text:


Natural language processing is an important field of artificial intelligence.
Natural language processing helps computers understand human language.
We are learning natural language processing techniques.
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
Tokenized Words:

['natural', 'language', 'processing', 'is', 'an', 'important', 'field', 'of', 'artificial', 'intelligence', '.', 'natural', 'language', 'processing',
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

```
Generated Bigrams:

[('natural', 'language'), ('language', 'processing'), ('processing', 'is'), ('is', 'an'), ('an', 'important'), ('important', 'field'), ('field', 'of'), ('of', 'artificial')
```

```
Bigram Frequencies:

('natural', 'language') : 3
('language', 'processing') : 3
('processing', 'is') : 1
('is', 'an') : 1
('an', 'important') : 1
('important', 'field') : 1
('field', 'of') : 1
('of', 'artificial') : 1
('artificial', 'intelligence') : 1
('intelligence', '.') : 1
('.', 'natural') : 1
('processing', 'helps') : 1
('helps', 'computers') : 1
('computers', 'understand') : 1
('understand', 'human') : 1
('human', 'language') : 1
('language', '.') : 1
('.', 'we') : 1
('we', 'are') : 1
('are', 'learning') : 1
('learning', 'natural') : 1
('processing', 'techniques') : 1
('techniques', '.') : 1
```

```
Language Model:

{'natural': [('language', 3)], 'language': [('processing', 3), ('.', 1)], 'processing': [('is', 1), ('helps', 1), ('techniques', 1)], 'is': [('an', 1)],
```

```
Input Word: natural
Predicted Next Word: language
Probability: 1.0
Input Word: language
Predicted Next Word: processing
Probability: 0.75
Input Word: processing
Predicted Next Word: is
Probability: 0.3333333333333333
```

Dataset :

```
path = kagglehub.dataset_download("gupta24789/text8-word-embedding")

file_path = os.path.join(path, "text8")

Using Colab cache for faster access to the 'text8-word-embedding' dataset.
```

```
...
    Input: king
    Context: ('king',)
    Predicted Next Word: of
    Probability: 0.4166666666666667

    Input: natural
    Context: ('natural',)
    Predicted Next Word: law
    Probability: 0.19047619047619047

    Input: language
    Context: ('language',)
    Predicted Next Word: as
    Probability: 0.16666666666666666

    Input: science
    Context: ('science',)
    Predicted Next Word: of
    Probability: 0.23809523809523808
```

Conclusion

The N-gram language model is a simple probabilistic method used to predict the next word based on previous words. It works by calculating word sequence frequencies and probabilities from a given text. Although it has limitations like limited context handling, it forms the foundation for advanced language modeling techniques in NLP.

Github link :

https://github.com/rachanadixit/NLP/blob/main/NLP_Assig_3_138.ipynb

https://github.com/rachanadixit/NLP/blob/main/Assi3_NLP_138_Dataset.ipynb